Assignment 5: Quicksort Algorithm: Implementation, Analysis, and Randomization

Haris Asir Hasan
MSCS 532-M70: Algorithms and Data Structures
Dr. Michael Solomon
October 17, 2026

Implementation

The deterministic implementation follows the Chapter 7 partition procedure (Cormen et al., 2022). For each active subarray, it selects the last element as the pivot. A scan then moves values less than or equal to the pivot to the low side. The final swap places the pivot between the low and high sides. The implementation recursively sorts the smaller side and then loops over the larger side. This preserves the Quicksort partitioning process while preventing one-sided partitions from exhausting Python's call stack. The implementation changes the input list in place and returns the same list so the caller can confirm that behavior.

The randomized version shares the same partition code. Its only algorithmic change is to choose an index uniformly from the current subarray and swap that value into the last position before partitioning. Passing one random-number generator through every recursive call permits reproducible tests without changing the pivot rule. It uses the same smaller-side recursion and larger-side loop as the deterministic version.

Both versions correctly handle empty lists, one-element lists, duplicate values, negative values, sorted input, and reverse-sorted input. Regression tests cover 1,200-element sorted, reverse-sorted, and equal-value lists, and the suite also checks 101 generated input sizes. Each function uses descriptive names, type hints, docstrings, and comments at the partition and recursion steps.

Theoretical Performance

Partitioning examines each element in the active subarray once, so one partition costs Theta(n). If the pivot splits the input into two nearly equal parts, the recurrence is T(n) = 2T(n/2) + Theta(n), which solves to Theta(n log n). This is the best case.

When input order is not related to consistently poor pivots, partitions vary in size. Across the recursion tree, there are normally Theta(log n) levels with Theta(n) total partition work per level. This gives an average-case time of O(n log n). The expected bound for randomized Quicksort follows the same idea because each position in the active subarray has an equal chance of becoming the pivot.

The worst case occurs when each pivot is the smallest or largest active value. One side then has n - 1 elements and the other side is empty. The recurrence becomes T(n) = T(n - 1) + Theta(n), and the accumulated partition work is Theta(n squared). A last-element pivot causes this behavior on already sorted and reverse-sorted lists. Randomization does not remove the theoretical worst case, but it makes the pivot sequence independent of the original input order and makes repeated extreme splits unlikely.

Partitioning itself uses constant auxiliary storage. Because each function recurses only on the smaller partition, the recursive side contains at most half of the current elements. Python call-stack depth is therefore O(log n) even when the loop repeatedly handles one-sided partitions. The randomized version also performs a random-index selection and swap per partition and keeps one generator object. These are constant-size overheads, although Python function calls and random number generation add measurable time.

Empirical Method

The benchmark used Python 3.11.16 on the current machine. It tested 100, 250, 500, and 750 distinct integers in random, sorted, and reverse-sorted order. Each algorithm, size, and distribution ran nine times. The reported value is the median elapsed time from the high-resolution performance counter. Input construction and list copying occurred before timing. Every one of the 216 timed runs had to match Python's sorted result or the benchmark stopped with an error.

Results

Distribution	n	Fixed pivot median (ms)	Random pivot median (ms)
Random	100	0.033	0.051
Random	250	0.089	0.136
Random	500	0.214	0.302
Random	750	0.345	0.499
Sorted	100	0.192	0.050
Sorted	250	1.132	0.135
Sorted	500	4.657	0.304
Sorted	750	10.569	0.457
Reverse sorted	100	0.132	0.051
Reverse sorted	250	0.762	0.138
Reverse sorted	500	3.170	0.303
Reverse sorted	750	7.349	0.457

Table 1. Median elapsed time from nine trials. Times measure only the sorting call.

Results and Discussion

On random input, the fixed-pivot version was faster at every tested size. At n = 750, its median was 0.345 ms compared with 0.499 ms for the randomized version. The additional random selection and swap cost about 44.8 percent at that point. Both versions still showed growth consistent with the expected n log n behavior over this limited range.

Input order changed the deterministic result sharply. On sorted input, its median rose from 0.192 ms at n = 100 to 10.569 ms at n = 750, a factor of about 54.9 while n increased by a factor of 7.5. A quadratic curve would increase by a factor of 56.25 over the same size change. Reverse-sorted input produced a similar pattern, rising from 0.132 ms to 7.349 ms.

The randomized version did not inherit the same ordered-input penalty. Its n = 750 medians were 0.457 ms for sorted input and 0.457 ms for reverse-sorted input. At that size it was about 23.1 times faster than the fixed-pivot version on sorted input and 16.1 times faster on reverse-sorted input. These measurements support the theoretical explanation: random pivots disconnect the recursion shape from an input order that is adversarial for the last-element rule.

Limitations

The measurements are short, machine-specific Python timings rather than a universal performance ranking. Background activity, interpreter behavior, and timer resolution can affect small results. The largest benchmark size was 750 to keep repeated quadratic fixed-pivot trials short. The empirical inputs used distinct integers to match the main Chapter 7 randomized analysis. Duplicate-heavy data can still create quadratic partition work under this two-way partition rule, but the smaller-side recursion strategy keeps the call stack bounded. Correctness tests include 1,200 equal values.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

OpenAI. (2026). *Codex* [Large language model; assisted with source organization, code development, testing, analysis, and drafting]. https://openai.com/codex/